

Planar Travel System - Technical Design

Project: DND35Prototype
System: Planar Travel & Gates
Date: May 25, 2026
Status: Design Phase

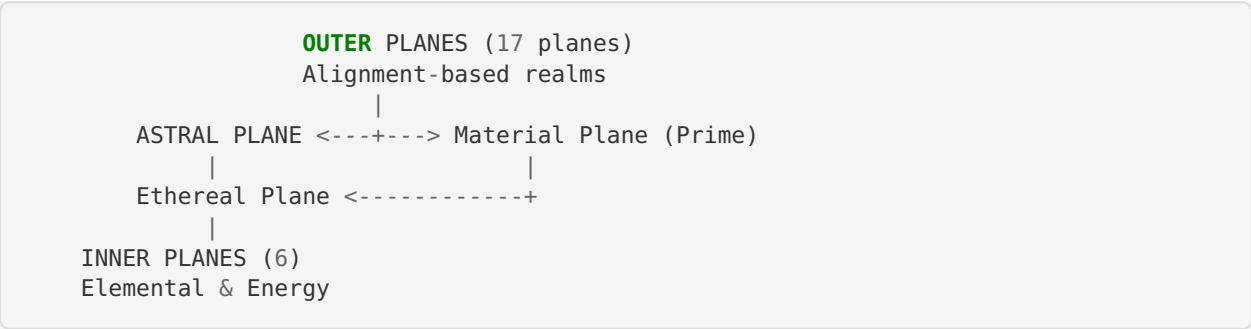
SYSTEM OVERVIEW

The Planar Travel System enables characters to travel between different planes of existence in the D&D 3.5e multiverse. This system is required for implementing several major wondrous items including the Amulet of the Planes, Cubic Gate, Well of Many Worlds, Robe of Stars, and Candle of Invocation.

- Core Capabilities:**
- Travel between planes (Plane Shift)
 - Create temporary gates/portals (Gate spell)
 - Handle environmental hazards per plane
 - Support both voluntary and involuntary travel
 - Track current plane for each character

D&D 3.5E PLANAR STRUCTURE

The Multiverse



Plane Categories

- 1. Material Plane (Prime Material)**
- The “normal” world where most campaigns take place
 - Standard physics and magic
- 2. Transitive Planes** (connect other planes)
- **Astral Plane:** Connects Material to Outer Planes
 - **Ethereal Plane:** Overlaps Material, connects to Inner Planes
 - **Plane of Shadow:** Parallel to Material, dark reflection

3. Inner Planes (elemental and energy)

- Elemental Air
- Elemental Earth
- Elemental Fire
- Elemental Water
- Positive Energy Plane
- Negative Energy Plane
- **Paraelemental Planes:** Ice, Magma, Ooze, Smoke
- **Quasielemental Planes:** Lightning, Mineral, Radiance, Steam, Vacuum, Ash, Dust, Salt

4. Outer Planes (alignment-based, 17 total)

- **Lawful Good:** Celestia (Seven Heavens), Bytopia, Arcadia
- **Neutral Good:** Elysium
- **Chaotic Good:** Arborea (Olympus)
- **Lawful Neutral:** Mechanus, Acheron
- **True Neutral:** Concordant Opposition (Outlands)
- **Chaotic Neutral:** Limbo
- **Lawful Evil:** Nine Hells (Baator)
- **Neutral Evil:** Gehenna, Hades
- **Chaotic Evil:** Abyss, Carceri, Pandemonium

SYSTEM ARCHITECTURE

Core Components

```

PlanarTravelSystem (static manager)
[ ] PlaneShift(Character, Plane, bool willSave)
[ ] CreateGate(Plane, duration, location)
[ ] CreatePortal(Plane, isTwoWay, location)
[ ] GetCurrentPlane(Character)

Plane (enum + data class)
[ ] PlaneType (enum: Material, Astral, etc.)
[ ] PlaneProperties (gravity, time, magic, etc.)
[ ] EnvironmentalHazards

PlaneEnvironment (per-plane effects)
[ ] GravityType (normal, heavy, light, none, subjective)
[ ] TimeFlow (normal, erratic, flowing)
[ ] MagicTraits (normal, enhanced, impeded, limited)
[ ] Hazards (fire damage, negative energy, etc.)

PlanarPortal (game object)
[ ] DestinationPlane
[ ] IsTwoWay
[ ] Duration
[ ] OnEnter(Character)
  
```

PLANE ENUM

```

public enum Plane
{
    // Material
    Material,

    // Transitive
    Astral,
    Ethereal,
    Shadow,

    // Inner Planes - Elemental
    Elemental_Air,
    Elemental_Earth,
    Elemental_Fire,
    Elemental_Water,

    // Inner Planes - Energy
    Positive,
    Negative,

    // Inner Planes - Paraelemental
    Paraelemental_Ice,
    Paraelemental_Magma,
    Paraelemental_Ooze,
    Paraelemental_Smoke,

    // Inner Planes - Quasielemental
    Quasielemental_Lightning,
    Quasielemental_Mineral,
    Quasielemental_Radiance,
    Quasielemental_Steam,
    Quasielemental_Vacuum,
    Quasielemental_Ash,
    Quasielemental_Dust,
    Quasielemental_Salt,

    // Outer Planes - Lawful Good
    Celestia,
    Bytopia,
    Arcadia,

    // Outer Planes - Neutral Good
    Elysium,

    // Outer Planes - Chaotic Good
    Arborea,

    // Outer Planes - Lawful Neutral
    Mechanus,
    Acheron,

    // Outer Planes - True Neutral
    Outlands,

    // Outer Planes - Chaotic Neutral
    Limbo,

    // Outer Planes - Lawful Evil
    Nine_Hells,

    // Outer Planes - Neutral Evil
    Gehenna,

```

```
Hades,  
  
// Outer Planes - Chaotic Evil  
Abyss,  
Carceri,  
Pandemonium  
}
```

PLANE PROPERTIES

PlaneData Class

```

[Serializable]
public class PlaneData
{
    public Plane Plane;
    public string Name;
    public string Description;

    // Physical traits
    public GravityType Gravity;
    public TimeFlowType TimeFlow;
    public SizeType Size; // Finite, Infinite
    public MorphicType Morphic; // Alterable, Divinely Morphic, etc.

    // Magic traits
    public MagicType Magic;
    public List<SchoolEnhancement> EnhancedSchools;
    public List<SchoolImpediment> ImpededSchools;

    // Alignment traits
    public AlignmentTrait AlignmentTrait; // Mildly/Strongly aligned

    // Environmental hazards
    public List<EnvironmentalHazard> Hazards;
}

public enum GravityType
{
    Normal,          // Standard 1g
    Heavy,           // 2x weight, -2 Str/Dex checks
    Light,           // Jump double distance
    None,            // Weightless, fly speed
    Objective_Directional, // Down is always down
    Subjective_Directional // Down is where you think it is
}

public enum TimeFlowType
{
    Normal,          // 1 hour = 1 hour
    Erratic,         // DM rolls each time
    Flowing,         // 1 day = 1 hour on Material
    Timeless         // No aging, no healing, no hunger
}

public enum MagicType
{
    Normal,
    Enhanced,        // +2 CL for certain schools
    Impeded,         // -2 CL for certain schools
    Limited,         // Some magic doesn't work
    Wild,            // Magic unpredictable
    Dead             // No magic at all
}

public enum AlignmentTrait
{
    None,
    MildlyAligned,   // -2 penalty on Cha checks for opposed alignment
    StronglyAligned  // -2 penalty on all checks for opposed alignment
}

```

PLANE DATABASE

```

public static class PlaneDatabase
{
    private static Dictionary<Plane, PlaneData> planes;

    static PlaneDatabase()
    {
        planes = new Dictionary<Plane, PlaneData>
        {
            {
                Plane.Material,
                new PlaneData
                {
                    Plane = Plane.Material,
                    Name = "Material Plane",
                    Description = "The world of mortals",
                    Gravity = GravityType.Normal,
                    TimeFlow = TimeFlowType.Normal,
                    Size = SizeType.Infinite,
                    Morphic = MorphicType.Alterable,
                    Magic = MagicType.Normal,
                    AlignmentTrait = AlignmentTrait.None,
                    Hazards = new List<EnvironmentalHazard>()
                }
            },
            {
                Plane.Elemental_Fire,
                new PlaneData
                {
                    Plane = Plane.Elemental_Fire,
                    Name = "Elemental Plane of Fire",
                    Description = "An endless inferno",
                    Gravity = GravityType.Objective_Directional,
                    TimeFlow = TimeFlowType.Normal,
                    Size = SizeType.Infinite,
                    Magic = MagicType.Enhanced,
                    EnhancedSchools = new List<SchoolEnhancement> { SchoolEnhance-
ment.Evocation_Fire },
                    ImpededSchools = new List<SchoolImpediment> { SchoolImpedi-
ment.Evocation_Water },
                    Hazards = new List<EnvironmentalHazard>
                    {
                        new EnvironmentalHazard
                        {
                            Type = HazardType.ExtremeHeat,
                            Damage = "3d10 per round",
                            FortSaveDC = 15,
                            Description = "Overwhelming heat deals fire damage each
round"
                        }
                    }
                }
            },
            {
                Plane.Negative,
                new PlaneData
                {
                    Plane = Plane.Negative,
                    Name = "Negative Energy Plane",
                    Description = "The void where life ends",
                    Gravity = GravityType.Subjective_Directional,
                    TimeFlow = TimeFlowType.Normal,
                    Size = SizeType.Infinite,

```

```

        Magic = MagicType.Enhanced,
        EnhancedSchools = new List<SchoolEnhancement> { SchoolEnhance-
ment.Necromancy },
        Hazards = new List<EnvironmentalHazard>
        {
            new EnvironmentalHazard
            {
                Type = HazardType.NegativeEnergy,
                Damage = "1d6 per round",
                Description = "Major negative-dominant: gain 1 negative
level per round"
            }
        }
    },
    {
        Plane.Nine_Hells,
        new PlaneData
        {
            Plane = Plane.Nine_Hells,
            Name = "Nine Hells of Baator",
            Description = "The ordered realm of devils",
            Gravity = GravityType.Normal,
            TimeFlow = TimeFlowType.Normal,
            Size = SizeType.Infinite,
            Magic = MagicType.Normal,
            AlignmentTrait = AlignmentTrait.StronglyAligned,
            AlignmentRequired = Alignment.LawfulEvil,
            Hazards = new List<EnvironmentalHazard>()
        }
    },
    // ... more planes
};

public static PlaneData GetPlaneData(Plane plane)
{
    return planes.ContainsKey(plane) ? planes[plane] : null;
}
}

```

ENVIRONMENTAL HAZARDS

```

[Serializable]
public class EnvironmentalHazard
{
    public HazardType Type;
    public string Damage; // Dice expression or "1 negative level"
    public int FortSaveDC;
    public string Description;
    public int FrequencyRounds; // How often damage occurs
}

public enum HazardType
{
    ExtremeHeat,      // Fire plane
    ExtremeCold,      // Ice plane
    NegativeEnergy,   // Negative Energy plane
    PositiveEnergy,   // Positive Energy plane (too much life)
    Vacuum,           // Quasielemental Vacuum
    Acid,             // Ooze plane
    Lava,             // Magma plane
    Lightning,        // Lightning plane
    Drowning          // Water plane (if not aquatic)
}

public class EnvironmentalHazardManager
{
    public static void ApplyPlaneHazards(Character character, Plane plane)
    {
        PlaneData data = PlaneDatabase.GetPlaneData(plane);

        foreach (var hazard in data.Hazards)
        {
            // Check if character has protection
            if (IsProtectedFrom(character, hazard.Type))
                continue;

            // Apply hazard
            switch (hazard.Type)
            {
                case HazardType.ExtremeHeat:
                    if (!SavingThrows.MakeFortSave(character, hazard.FortSaveDC))
                    {
                        int damage = DiceRoller.Roll(hazard.Damage);
                        character.TakeDamage(damage, DamageType.Fire);
                    }
                    break;

                case HazardType.NegativeEnergy:
                    character.GainNegativeLevel(1);
                    break;

                // ... more hazard types
            }
        }
    }

    private static bool IsProtectedFrom(Character character, HazardType hazard)
    {
        switch (hazard)
        {
            case HazardType.ExtremeHeat:
                return character.HasSpellEffect("Resist Energy (Fire)") ||
                    character.HasSpellEffect("Protection from Energy (Fire)");
        }
    }
}

```

```
        case HazardType.NegativeEnergy:
            return character.HasSpellEffect("Death Ward") ||
                character.Type == CreatureType.Undead;

        // ... more protections
    }

    return false;
}
```

PLANAR TRAVEL SYSTEM

PlanarTravelSystem.cs

```

public static class PlanarTravelSystem
{
    // Store current plane for each character
    private static Dictionary<int, Plane> characterPlanes = new Dictionary<int,
Plane>();

    /// <summary>
    /// Teleport character to another plane (Plane Shift spell)
    /// </summary>
    public static bool PlaneShift(Character traveler, Plane destination, bool re-
quiresWillSave = true, int saveDC = 20)
    {
        // Willing save check
        if (requiresWillSave && !traveler.IsWilling)
        {
            if (SavingThrows.MakeWillSave(traveler, saveDC))
            {
                CombatLog.Add($"{traveler.Name} resists the plane shift!");
                return false;
            }
        }

        Plane oldPlane = GetCurrentPlane(traveler);

        // Change plane
        characterPlanes[traveler.ID] = destination;

        CombatLog.Add($"{traveler.Name} shifts from {oldPlane} to {destination}!");

        // Apply environmental effects
        ApplyPlanarEnvironment(traveler, destination);

        // Visual effect
        PlayPlanarShiftEffect(traveler.transform.position);

        return true;
    }

    /// <summary>
    /// Create temporary gate to another plane (Gate spell)
    /// </summary>
    public static PlanarGate CreateGate(Plane destinationPlane, int durationRounds, Vec
tor3 location)
    {
        GameObject gateObj = Instantiate(Resources.Load<GameObject>("Prefabs/Planar-
Gate"), location, Quaternion.identity);
        PlanarGate gate = gateObj.GetComponent<PlanarGate>();

        gate.DestinationPlane = destinationPlane;
        gate.Duration = durationRounds;
        gate.IsTwoWay = true;
        gate.AllowsCalling = true; // Gate can summon creatures

        StartCoroutine(DestroyGateAfterDuration(gate, durationRounds));

        return gate;
    }

    /// <summary>
    /// Create portal (like Well of Many Worlds)
    /// </summary>
    public static PlanarPortal CreatePortal(Plane destinationPlane, bool isTwoWay, Vec

```

```

tor3 location)
{
    GameObject portalObj = Instantiate(Resources.Load<GameObject>("Prefabs/Planar-Portal"), location, Quaternion.identity);
    PlanarPortal portal = portalObj.GetComponent<PlanarPortal>();

    portal.DestinationPlane = destinationPlane;
    portal.IsTwoWay = isTwoWay;
    portal.AllowsCalling = false; // Portals don't summon

    return portal;
}

/// <summary>
/// Get character's current plane
/// </summary>
public static Plane GetCurrentPlane(Character character)
{
    if (characterPlanes.ContainsKey(character.ID))
        return characterPlanes[character.ID];

    return Plane.Material; // Default to Material Plane
}

/// <summary>
/// Apply environmental effects of plane
/// </summary>
private static void ApplyPlanarEnvironment(Character character, Plane plane)
{
    PlaneData data = PlaneDatabase.GetPlaneData(plane);

    // Apply gravity changes
    ApplyGravity(character, data.Gravity);

    // Apply magic traits
    ApplyMagicTraits(character, data.Magic, data.EnhancedSchools, data.ImpededSchools);

    // Apply alignment penalties
    ApplyAlignmentPenalties(character, data.AlignmentTrait, data.AlignmentRequired);

    // Start hazard loop (damage over time)
    if (data.Hazards.Count > 0)
    {
        StartCoroutine(ApplyHazardsOverTime(character, plane));
    }
}

private static void ApplyGravity(Character character, GravityType gravity)
{
    switch (gravity)
    {
        case GravityType.Normal:
            character.GravityMultiplier = 1.0f;
            break;

        case GravityType.Heavy:
            character.GravityMultiplier = 2.0f;
            character.AddPenalty("Heavy Gravity", -2, "Strength checks");
            character.AddPenalty("Heavy Gravity", -2, "Dexterity checks");
            break;
    }
}

```



```

        case GravityType.Light:
            character.GravityMultiplier = 0.5f;
            character.JumpDistanceMultiplier = 2.0f;
            break;

        case GravityType.None:
            character.GravityMultiplier = 0.0f;
            character.FlySpeed = 30; // Everyone can "fly" by pushing off
            break;

        case GravityType.Subjective_Directional:
            character.GravityMultiplier = 0.0f;
            character.CanChooseGravityDirection = true;
            break;
    }
}

private static void ApplyMagicTraits(Character character, MagicType magic, List<SchoolEnhancement> enhanced, List<SchoolImpediment> impeded)
{
    character.PlanarMagicType = magic;

    switch (magic)
    {
        case MagicType.Enhanced:
            foreach (var school in enhanced)
            {
                character.AddCasterLevelBonus(school, 2);
            }
            break;

        case MagicType.Impeded:
            foreach (var school in impeded)
            {
                character.AddCasterLevelPenalty(school, -2);
            }
            break;

        case MagicType.Dead:
            character.CanCastSpells = false;
            character.MagicItemsSuppressed = true;
            break;
    }
}

private static IEnumerator ApplyHazardsOverTime(Character character, Plane plane)
{
    while (GetCurrentPlane(character) == plane && character.IsAlive)
    {
        EnvironmentalHazardManager.ApplyPlaneHazards(character, plane);
        yield return new WaitForSeconds(6.0f); // 1 round = 6 seconds
    }
}
}

```

PLANAR GATE COMPONENT

```

public class PlanarGate : MonoBehaviour
{
    public Plane DestinationPlane;
    public int Duration; // rounds
    public bool IsTwoWay;
    public bool AllowsCalling; // Can summon creatures from plane

    private float remainingTime;

    void Start()
    {
        remainingTime = Duration * 6.0f; // rounds to seconds
    }

    void Update()
    {
        remainingTime -= Time.deltaTime;

        if (remainingTime <= 0)
        {
            CloseGate();
        }
    }

    void OnTriggerEnter(Collider other)
    {
        Character character = other.GetComponent<Character>();
        if (character != null)
        {
            TravelThroughGate(character);
        }
    }

    private void TravelThroughGate(Character traveler)
    {
        Plane currentPlane = PlanarTravelSystem.GetCurrentPlane(traveler);

        if (currentPlane == DestinationPlane && IsTwoWay)
        {
            // Traveling back through gate
            PlanarTravelSystem.PlaneShift(traveler, Plane.Material, requiresWillSave:
false);
        }
        else
        {
            // Traveling to destination
            PlanarTravelSystem.PlaneShift(traveler, DestinationPlane, requiresWill-
Save: false);
        }
    }

    public void SummonCreature(CreatureType creatureType)
    {
        if (!AllowsCalling)
            return;

        // Gate spell can call specific creatures from plane
        Character summoned = CreatureDatabase.SummonCreature(creatureType, Destina-
tionPlane);
        summoned.transform.position = transform.position;
        summoned.SummonedBy = gateCreator;
    }
}

```

```

private void CloseGate()
{
    PlayClosingEffect();
    Destroy(gameObject);
}
}

```

ITEM IMPLEMENTATIONS

Amulet of the Planes

```

public class AmuletOfThePlanes : WondrousItem
{
    public AmuletOfThePlanes()
    {
        Name = "Amulet of the Planes";
        Price = 120000;
        Slot = EquipmentSlot.Neck;
        CasterLevel = 15;
    }

    public void UsePlaneShift(Character wearer, Plane destination)
    {
        // At-will activation (no charges)
        PlanarTravelSystem.PlaneShift(wearer, destination, requiresWillSave: false, saveDC: 20);

        CombatLog.Add($"{wearer.Name} uses the Amulet of the Planes to travel to {destination}!");
    }
}

```

Cubic Gate

```

public class CubicGate : WondrousItem
{
    public Dictionary<int, Plane> SideAttunements;
    public Dictionary<int, int> SideCharges; // 3 per week per side

    public CubicGate()
    {
        Name = "Cubic Gate";
        Price = 164000;
        Slot = EquipmentSlot.Slotless;
        CasterLevel = 13;

        // Initialize 6 sides with attunements
        SideAttunements = new Dictionary<int, Plane>
        {
            { 1, Plane.Elemental_Fire },
            { 2, Plane.Elemental_Water },
            { 3, Plane.Elemental_Air },
            { 4, Plane.Elemental_Earth },
            { 5, Plane.Astral },
            { 6, Plane.Nine_Hells }
        };

        // Initialize charges (3/week per side)
        SideCharges = new Dictionary<int, int>();
        for (int i = 1; i <= 6; i++)
        {
            SideCharges[i] = 3;
        }
    }

    public void ActivateSide(int side, Vector3 location)
    {
        if (side < 1 || side > 6)
        {
            CombatLog.Add("Invalid side!");
            return;
        }

        if (SideCharges[side] <= 0)
        {
            CombatLog.Add($"Side {side} has no charges remaining this week!");
            return;
        }

        Plane destination = SideAttunements[side];

        // Create gate (10 rounds duration)
        PlanarGate gate = PlanarTravelSystem.CreateGate(destination, duration: 10, location);

        SideCharges[side]--;

        CombatLog.Add($"Cubic Gate opens to {destination}! (Side {side}: {SideCharges[side]} charges remaining)");
    }

    public void RestoreWeeklyCharges()
    {
        for (int i = 1; i <= 6; i++)
        {
            SideCharges[i] = 3;
        }
    }
}

```

```
    }  
    CombatLog.Add("Cubic Gate charges restored!");  
  }  
}
```

Well of Many Worlds


```

public class WellOfManyWorlds : WondrousItem
{
    private static Plane[] possiblePlanes =
    {
        Plane.Ethereal,
        Plane.Astral,
        Plane.Elemental_Air,
        Plane.Elemental_Earth,
        Plane.Elemental_Fire,
        Plane.Elemental_Water,
        Plane.Positive,
        Plane.Negative
    };

    public WellOfManyWorlds()
    {
        Name = "Well of Many Worlds";
        Price = 82000;
        Slot = EquipmentSlot.Slotless;
        CasterLevel = 17;
    }

    public PlanarPortal SpreadOnGround(Vector3 location)
    {
        // Random plane
        Plane randomPlane = possiblePlanes[Random.Range(0, possiblePlanes.Length)];

        // Check for Portable Hole nearby (within 10 ft)
        if (DetectPortableHoleNearby(location, 10.0f))
        {
            DestroyBothItems();
            CreateAstralGate(location);
            return null;
        }

        // Create two-way portal
        PlanarPortal portal = PlanarTravelSystem.CreatePortal(randomPlane, isTwoWay: true, location);

        CombatLog.Add($"Well of Many Worlds opens a portal to {randomPlane}!");

        return portal;
    }

    private bool DetectPortableHoleNearby(Vector3 location, float radius)
    {
        Collider[] nearbyObjects = Physics.OverlapSphere(location, radius);

        foreach (var obj in nearbyObjects)
        {
            WondrousItem item = obj.GetComponent<WondrousItem>();
            if (item != null && item.Name == "Portable Hole")
            {
                return true;
            }
        }

        return false;
    }

    private void DestroyBothItems()
    {

```

```

CombatLog.Add("CATASTROPHIC INTERACTION! Well of Many Worlds and Portable Hole
annihilate each other!");

    // Destroy both items
    Destroy(this.gameObject);
    // Portable Hole also destroyed by its own detection
}

private void CreateAstralGate(Vector3 location)
{
    // Creates permanent one-way gate to Astral Plane
    PlanarPortal astralGate = PlanarTravelSystem.CreatePortal(Plane.Astral, isTwo-
Way: false, location);
    astralGate.IsPermanent = true;

    CombatLog.Add("A rift to the Astral Plane tears open!");
}
}

```

Robe of Stars (Astral Travel)

```

public class RobeOfStars : WondrousItem
{
    public int StarsRemaining = 6;

    public void TravelThroughAstralPlane(Character wearer, Vector3 destination)
    {
        // Move to Astral Plane
        PlanarTravelSystem.PlaneShift(wearer, Plane.Astral, requiresWillSave: false);

        // In Astral, time passes differently (can cover vast distances quickly)
        // Simplified: teleport to destination on Material Plane after 1 round

        StartCoroutine(ReturnFromAstral(wearer, destination));
    }

    private IEnumerator ReturnFromAstral(Character wearer, Vector3 destination)
    {
        yield return new WaitForSeconds(6.0f); // 1 round

        // Return to Material at destination
        PlanarTravelSystem.PlaneShift(wearer, Plane.Material, requiresWillSave: false)
;
        wearer.transform.position = destination;

        CombatLog.Add($"{wearer.Name} emerges from the Astral Plane!");
    }
}

```

SAVE/LOAD PERSISTENCE

```

[Serializable]
public class PlanarTravelSaveData
{
    public Dictionary<int, Plane> CharacterPlanes;
    public List<ActiveGateSaveData> ActiveGates;
    public List<ActivePortalSaveData> ActivePortals;
}

[Serializable]
public class ActiveGateSaveData
{
    public Plane Destination;
    public Vector3 Location;
    public float RemainingTime;
    public bool IsTwoWay;
}

public static class PlanarTravelSaveLoad
{
    public static PlanarTravelSaveData SaveData()
    {
        PlanarTravelSaveData data = new PlanarTravelSaveData();

        // Save character planes
        data.CharacterPlanes = new Dictionary<int, Plane>(PlanarTravelSystem.characterPlanes);

        // Save active gates
        data.ActiveGates = new List<ActiveGateSaveData>();
        foreach (var gate in FindObjectsOfType<PlanarGate>())
        {
            data.ActiveGates.Add(new ActiveGateSaveData
            {
                Destination = gate.DestinationPlane,
                Location = gate.transform.position,
                RemainingTime = gate.remainingTime,
                IsTwoWay = gate.IsTwoWay
            });
        }

        return data;
    }

    public static void LoadData(PlanarTravelSaveData data)
    {
        // Restore character planes
        PlanarTravelSystem.characterPlanes = data.CharacterPlanes;

        // Recreate gates
        foreach (var gateData in data.ActiveGates)
        {
            PlanarGate gate = PlanarTravelSystem.CreateGate(
                gateData.Destination,
                (int)(gateData.RemainingTime / 6.0f),
                gateData.Location
            );
            gate.remainingTime = gateData.RemainingTime;
        }
    }
}

```

TESTING CHECKLIST

Basic Travel

- ☐ Character can plane shift to all defined planes
- ☐ Current plane tracked correctly
- ☐ Visual effects play on plane shift
- ☐ Multiple characters can be on different planes

Environmental Effects

- ☐ Fire plane deals fire damage
- ☐ Negative Energy plane grants negative levels
- ☐ Gravity changes apply correctly (heavy, light, none, subjective)
- ☐ Magic enhancement/impediment works
- ☐ Alignment penalties apply on aligned planes

Gates & Portals

- ☐ Gate opens with correct duration
- ☐ Characters can travel through gate
- ☐ Two-way gates work both directions
- ☐ Gates close after duration expires
- ☐ Portals remain until removed

Item-Specific

- ☐ Amulet of Planes: at-will travel works
- ☐ Cubic Gate: 6 sides, 3 charges each, correct planes
- ☐ Cubic Gate: charges restore weekly
- ☐ Well: random plane selected
- ☐ Well + Portable Hole: both destroyed, Astral gate created
- ☐ Robe of Stars: Astral travel teleports correctly

Edge Cases

- ☐ Character dies on hazardous plane
- ☐ Protection spells prevent hazard damage
- ☐ Gate destroyed before duration ends
- ☐ Multiple gates open simultaneously
- ☐ Save/load preserves planes and gates

PERFORMANCE CONSIDERATIONS

Potential Issues:

1. **Hazard damage loop** runs every round for every character on hazardous plane
2. **Multiple active gates** with collision detection
3. **Plane database** lookups per frame

Optimizations:

- Cache PlaneData for current plane
 - Use coroutines for hazard damage (not Update())
 - Pool gate/portal game objects
 - Limit number of simultaneous gates (e.g., max 5)
-

FUTURE ENHANCEMENTS

Phase 2 Additions:

- Coexistent planes (Ethereal overlaps Material)
 - Plane-specific creatures (demons in Abyss, etc.)
 - Tuning fork attunement (plane shift requires correct fork)
 - Planar boundaries (border zones between planes)
 - Color pools (portals in Astral Plane)
 - Planar traits affecting spells (e.g., Teleport works differently per plane)
-

ESTIMATED EFFORT

Implementation Time: 1-2 weeks

Breakdown:

- Plane enum + database: 2 days
- Core travel mechanics: 2 days
- Environmental hazards: 2 days
- Gate/portal objects: 2 days
- Item implementations: 1 day
- Testing + bug fixes: 2 days

Total: 11 days (~2 weeks)

Document Version: 1.0

Last Updated: May 25, 2026

Status: Ready for Implementation